



QUICKCOURT

Local Sports Booking Platform

Project Documentation

Project	Details
Type	Hackathon Pre-Project
Problem Statement	Problem Statement 1 - QuickCourt: A Local Sports Booking
Hackathon	ODDO Hackathon
Institution	JG University
Submission	21 September 2026
Development	Visual Studio Code
API Testing	Postman
Version Control	Git / GitHub

Prepared for college submission and GitHub documentation. Covers the implemented features, architecture, major APIs, technology stack, testing approach and team responsibilities.

1. Project Overview

QuickCourt is a full-stack web platform for discovering and booking local sports facilities such as badminton courts, turf grounds, tennis tables and other sports venues. It provides an end-to-end booking experience for players and dedicated management workflows for Facility Owners and Administrators.

The project was developed as a pre-project for the ODDO Hackathon at JG University and follows the supplied QuickCourt problem statement.

Objectives

- Simplify local sports venue discovery and booking.
- Provide court, date, time-slot and price selection.
- Allow facility owners to manage facilities, courts, availability and bookings.
- Provide administrators with facility approval and user/platform management.
- Use React, Spring Boot and MySQL in a separated full-stack architecture.
- Test backend REST APIs using Postman.

2. Roles & Major Features

Player

- Home, popular venues/sports and search.
- Approved venue listing with search, sport/price/venue/rating filters and pagination where implemented.
- Venue details: sports, amenities, address, description and gallery/review areas where implemented.
- Court, date and time-slot selection.
- Price/total review and simulated payment/confirmation.
- My Bookings with Confirmed, Cancelled and Completed states.
- Profile and matches/community functionality where implemented.

Facility Owner

- Owner dashboard and KPIs.
- Facility add/edit/manage workflow.
- Court management: name, sport, price and operating information.
- Availability and maintenance time-slot management.
- Upcoming/past booking overview.
- Owner profile and analytics where implemented.

Administrator

- Global dashboard and platform statistics.
- Pending facility review and approval/rejection.
- User and facility-owner management.
- Search/filter by role or status.
- Ban/unban and booking-history access where implemented.
- Reports/moderation structure and admin profile.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, JavaScript/JSX, CSS	Web application UI
Routing	React Router DOM	Client-side navigation
API Client	Axios	REST API communication
UI/Charts	Lucide React, Recharts	Icons and dashboard charts
Backend	Java, Spring Boot, Maven	REST APIs and business logic

Layer	Technology	Purpose
Security	Spring Security	Authentication/authorization
Persistence	Spring Data JPA / Hibernate	ORM and database access
Database	MySQL	Relational data storage
Testing	Postman	API testing
Development	Visual Studio Code	Development environment
Version Control	Git / GitHub	Source control

4. Frontend Implementation

The frontend is built with React and Vite. React Router DOM manages navigation and role-specific routes. Axios connects the frontend to Spring Boot REST APIs. Recharts is used for dashboard visualizations and Lucide React provides interface icons. CSS provides responsive layouts and application styling.

Main areas include authentication, home/venue discovery, venue details, booking, My Bookings, Profile, owner dashboard/management and admin dashboard/management.

```
frontend/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── pages/
│   ├── context/
│   ├── services/
│   ├── App.*
│   └── main.*
├── package.json
└── vite.config.*
```

5. Backend & Database

The backend uses Java and Spring Boot. Spring Security protects restricted resources, Spring Data JPA/Hibernate handles persistence, Maven manages dependencies/builds, and MySQL stores application data.

Request flow:

```
React UI → Axios → REST Controller → Business/Service Layer → Repository/JPA → MySQL
```

Major data domains include users/roles, facilities, courts, sports, availability/time slots, bookings and profile/administrative information.

6. APIs & Integration

QuickCourt uses REST APIs exposed by the Spring Boot backend. Axios is the frontend HTTP client. Postman was used to test the backend APIs during development.

API Area	Purpose	Used By
Authentication	Registration, login and authenticated access	All roles
User/Profile	Read/update profile and user information	Player, Owner, Admin
Facility/Venue	Venue discovery and facility management	Player, Owner, Admin
Court	Court creation, retrieval, update and deletion where supported	Owner/Admin
Availability/Slots	Court availability and blocked periods	Owner/Player
Booking	Create, retrieve and manage booking records	Player/Owner/Admin
Owner	Owner dashboard and management operations	Facility Owner
Admin	Approvals and user/platform administration	Administrator

Important: exact endpoint URLs, request bodies and response schemas are implementation-specific. They should be taken from the current backend controllers/Postman collection; this report does not invent endpoint paths.

REST methods used by the application follow the standard pattern: GET for retrieval, POST for creation/submission, PUT/PATCH for updates and DELETE for permitted removal operations.

7. Authentication & Booking Flow

Spring Security provides backend security and role-based authorization. The application separates Player, Facility Owner and Administrator workflows.

Booking flow

Player → Venue → Court → Date → Available Time Slot → Price Review → Confirmation / Simulated Payment → Booking Created → My Bookings

Booking-related operations are validated by the backend according to the implemented business rules and availability data.

8. API Testing

Postman was used as the primary API testing tool during development.

- Registration and login requests.
- Protected endpoint access.
- Facility and venue operations.
- Court operations.
- Availability/time-slot operations.
- Booking creation and retrieval.
- Owner-specific operations.
- Admin-specific operations.
- Request/response payload and HTTP status verification.
- Frontend-backend integration verification.

9. Setup & Deployment

Prerequisites

- Node.js and npm
- Java JDK
- Maven
- MySQL
- Git
- Postman
- Visual Studio Code

Frontend

```
cd frontend
npm install
npm run dev
```

Backend

```
cd backend
mvn spring-boot:run
```

Production builds

```
Frontend: npm run build
Backend: mvn clean package
```

Deployment model

```
React Frontend → Static Hosting
Spring Boot Backend → Java Backend Hosting
```

Production configuration must use the correct API URL, CORS origin and database credentials. Never commit passwords, JWT secrets or private credentials to GitHub.

10. Testing Checklist

- Registration / login / logout.
- Role-based access and protected routes.
- Venue search, filters and details.
- Court and time-slot selection.
- Booking creation, status and eligible cancellation.
- Owner facility/court/booking management.
- Admin facility approval and user management.
- Database connectivity.
- REST API responses using Postman.
- Frontend production build and backend package build.
- Responsive UI and route refresh behavior.

11. Team Members & Contributions

Member	Academic Details	GitHub	Contribution
Samir Prajapat	Roll: 241208 Enrollment: 2401061155	samir416	Complete backend, database, Spring Boot, REST APIs, Spring Security, JPA integration, authentication/authorization, business logic, API testing and backend integration.
Soham Ashar	Roll: 241006 Enrollment: 2400941004	sohamashar21	Complete frontend, React/Vite application, UI, routing, responsive design, components, dashboards, booking interfaces and API integration.
Aditya Solanki	Roll: 241212 Enrollment: 2400941139	aditya-solanki	Requirement analysis, documentation, feature mapping, functional testing/validation, integration review, presentation support and hackathon submission coordination.

12. Future Scope

- Real payment gateway and refund workflow.
- Real-time availability.
- Maps/location integration.
- Email, SMS and push notifications.
- Advanced ratings and reviews.
- Player matchmaking and match creation.
- Cloud image storage.
- Advanced analytics and revenue reports.
- Mobile application.
- Advanced moderation and automated booking reminders.

13. Conclusion

QuickCourt provides a practical full-stack solution for local sports venue discovery, court booking and facility management. The project combines a React/Vite frontend, Java/Spring Boot backend, Spring Security, Spring Data JPA, MySQL and REST API

integration.

The implementation addresses the main functional areas of the supplied ODDO Hackathon QuickCourt problem statement through Player, Facility Owner and Administrator workflows. Postman was used for API testing, while Git/GitHub supports source-code version control.

JG University • ODDO Hackathon Pre-Project • Submission: 21 September 2026